Concurrency and Distributed Systems

Week 5 – Part 4: Measurement, Atomics, Read-Write Locks, and ThreadLocal

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 4 Roadmap

1. Measurement and Observability
2. Atomics and CAS
3. Read-Write Locks
4. ThreadLocal Context
5. Checkpoint 2 Bridge
6. Lab Preparation

Why This Part Matters

Core question

Once work is running under a bounded policy, how do we observe and optimize shared state?

Week 5 ends by connecting bounded execution to:

- metrics,
- read-heavy shared state,
- per-request context,
- the Checkpoint 2 server.

Bounded Concurrency Needs Measurement

- Bounded queues tell us when work is waiting.
- Bounded pools tell us when workers are busy.
- Rejection tells us when capacity is exhausted.
- Metrics turn those events into operational evidence.

Engineering rule

A concurrency policy that is not measured is only a guess.

Measurement Lens

Metric	Question answered
Throughput	How many tasks complete per second?
Latency	How long does one task wait and run?
Queue depth	How much work is waiting?
Rejections	How often do we hit capacity?
Active workers	Is the pool saturated?
Success rate	Are tasks completing correctly?

Throughput vs Latency

- Throughput is aggregate capacity.
- Latency is individual experience.
- A larger queue can increase throughput while hurting latency.
- A cached pool can improve burst completion while increasing resource risk.

Week 5 measurement rule

Always report at least one throughput observation and one tradeoff paragraph.

Shared Metrics Problem

Scenario

Many worker threads complete requests and update counters at the same time.

- Total requests.
- Success count.
- Failure count.
- Success rate.

Question

How do we update those numbers without losing increments?

Demo07: Synchronized Counter

```
1 private int value;
2
3 public synchronized void increment() {
4     value++;
5 }
6
7 public synchronized int get() {
8     return value;
9 }
```

- Correct.
- Simple.
- All readers and writers use the same monitor.
- Under contention, threads serialize on one lock.

Demo08: Atomic Counter

```
1 private final AtomicInteger value = new AtomicInteger();
2
3 public int increment() {
4     return value.incrementAndGet();
5 }
6
7 public int get() {
8     return value.get();
9 }
```

- No explicit lock.
- Atomic operation provided by the JDK.
- Good fit for simple counters and flags.

Compare-And-Swap

CAS idea

Update a value only if it still equals the value you observed earlier.

```
compareAndSet(expected, update)
```

- If current value equals expected, update succeeds.
- If another thread changed it first, update fails.
- The caller can retry with a fresh observed value.

Demo09: CAS Retry Loop

```
while (true) {
    int current = value.get();
    int next = current + delta;
    if (value.compareAndSet(current, next)) {
        return next;
    }
}
```

What students should notice

CAS is optimistic: try the update, retry only if someone changed the value first.

Atomics: When They Fit

- Counters.
- Sequence numbers.
- Boolean flags.
- Single-reference swaps.
- Simple metrics under high contention.

Limit

Atomics are not a replacement for all invariants. Multi-field consistency often still needs a lock or immutable snapshot.

Exercise 3: AtomicMetrics

```
public void recordSuccess() { ... }
public void recordFailure() { ... }

public int successes() { ... }
public int failures() { ... }
public int total() { ... }
public double successRatePercent() { ... }
```

- Use atomic counters.
- Concurrent updates must not be lost.
- Empty metrics should report a zero success rate.

Read-Mostly State

Scenario

A server has cached configuration or lookup data: many reads, occasional writes.

- Full synchronization is correct.
- But it blocks readers behind other readers.
- Read-heavy workloads can often do better.

Demo10: Synchronized Cache

```
1 private final Map<K, V> values = new HashMap<>();
2
3 public synchronized void put(K key, V value) {
4     values.put(key, value);
5 }
6
7 public synchronized V get(K key) {
8     return values.get(key);
9 }
```

- Correct mutual exclusion.
- One operation at a time.
- Readers block readers.

Read-Write Lock

- Read lock: many readers may hold it together.
- Write lock: exclusive access.
- Good when reads dominate and reads are non-trivial.
- Not free: more complex than `synchronized`.

Course rule

Consider `ReentrantReadWriteLock` when read:write $>$ 10:1 and reads are meaningful work.

Demo11: ReadWriteLock Cache

```
lock.readLock().lock();
try {
    return values.get(key);
} finally {
    lock.readLock().unlock();
}

lock.writeLock().lock();
try {
    values.put(key, value);
} finally {
    lock.writeLock().unlock();
}
```

- Reads can run concurrently.
- Writes exclude all access.
- Unlock in `finally`.

When ReadWriteLock Is the Wrong Tool

- Reads are tiny and writes are common.
- The protected object has complex invariants.
- You need atomic update across several structures.
- Simpler synchronization is easier to audit.

Pragmatic advice

Use read-write locking for a measured read-heavy bottleneck, not as default style.

Exercise 4: ReadMostlyCache

```
public void put(K key, V value) { ... }
public V get(K key) { ... }
public int size() { ... }
public Map<K, V> snapshot() { ... }
```

- Use `ReentrantReadWriteLock`.
- Protect writes with the write lock.
- Protect reads and snapshots with the read lock.
- Return an immutable snapshot copy.

Request Context

Scenario

A server wants the current request ID available across helper methods without passing it everywhere.

- Logging correlation ID.
- User ID.
- Tenant ID.
- Request deadline.

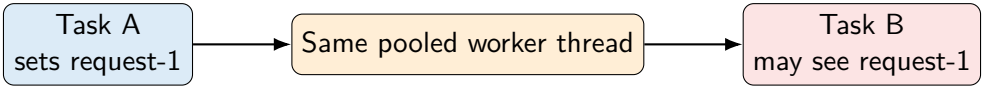
Demo12: ThreadLocal Context

```
private static final ThreadLocal<String> REQUEST_ID =
    new ThreadLocal<>();

REQUEST_ID.set(requestId);
try {
    return REQUEST_ID.get();
} finally {
    REQUEST_ID.remove();
}
```

- Each thread sees its own value.
- The value is attached to the worker thread.
- Cleanup is mandatory in pools.

ThreadLocal in a Thread Pool



Problem

Threads outlive requests. ThreadLocal values can outlive requests too.

Demo13: ThreadLocal Leak

```
public static String unsafeHandle(String requestId, boolean setValue) {
    if (setValue) {
        REQUEST_ID.set(requestId);
    }
    return REQUEST_ID.get();
}
```

- Task A sets a value.
- It does not remove it.
- Later work on the same thread can observe stale context.

Safe ThreadLocal Pattern

```
public static void runWithContext(String requestId, Runnable task) {
    REQUEST_ID.set(requestId);
    try {
        task.run();
    } finally {
        REQUEST_ID.remove();
    }
}
```

Rule

The same method that sets ThreadLocal context should own cleanup.

Exercise 5: RequestContext

```
1 public static void set(String requestId) { ... }
2 public static String get() { ... }
3 public static void clear() { ... }
4 public static void runWithContext(String requestId, Runnable task) { ... }
```

- Store data per thread.
- Prove isolation between threads.
- Prove cleanup after task execution.
- Use `try/finally`.

Checkpoint 2 Bridge

Week 5 exercise	Server component
Ex1 BoundedBuffer	Request queue.
Ex2 BoundedExecutor	Server worker pool.
Ex3 AtomicMetrics	Request counts and success rate.
Ex4 ReadMostlyCache	Read-heavy cache or configuration layer.
Ex5 RequestContext	Per-request ID and cleanup.

Checkpoint 2 Expectations

- Bounded resources: no unbounded queues, no thread-per-request explosion.
- Correctness: no lost metrics updates, no stale request context.
- Liveness note: what can block and how it unblocks.
- Measurement table: throughput or timing observation.
- Tradeoff paragraph: explain selected pool and queue policy.

Lab Preparation

- Read `Demo07`–`Demo09` before Ex3.
- Read `Demo10`–`Demo11` before Ex4.
- Read `Demo12`–`Demo13` before Ex5.
- Complete the scorecard after implementation, not before.

Week 5 Takeaway

Final takeaway

Bounded concurrency means every shared resource has an explicit capacity, observation point, and overload behavior.

Bridge to next week

Week 6 moves from managing tasks to decomposing computation:

parallelism and Fork/Join.

End of Week 5

Questions?